Índices de vecinos más cercanos aproximados

Compromiso exactitud–tiempo–memoria de LSH, IVF-PQ y HNSW

David Segundo García

Actividad 3A · 28 de septiembre de 2026 · acompaña al notebook `Actividad3A_Indices_ANN.ipynb`

1. Montaje experimental

Colecciones. *Embeddings* de palabras **GloVe 6B** de 100 dimensiones (Stanford NLP; Wikipedia 2014 + Gigaword 5). De las 101 000 palabras más frecuentes se apartaron 1 000 al azar como consultas no indexadas. Las 100 000 restantes forman la colección **100k** (38.15 MB) y una muestra aleatoria de ellas la colección **10k** (3.81 MB), con 10k $\subset$ 100k y la misma distribución. Los vectores son `float32` con norma $L_2 = 1$.

Métrica y línea base. Se usa la **distancia coseno**, porque en los *embeddings* de palabras el significado está en la dirección y la norma refleja la frecuencia. Con vectores unitarios, $\|x - y\|^2 = 2\, d_{\cos}(x, y)$, así que L_2, producto interno y coseno dan el mismo orden de vecinos. Eso permite usar índices L_2 (IVF-PQ) y es la hipótesis del LSH de hiperplanos (SimHash). La referencia exacta es una fuerza bruta con NumPy, verificada contra `faiss.IndexFlatIP` con concordancia de 1.000, y de ella sale el *ground truth* del recall@10.

Índices y medición. Se implementaron las tres familias y se barrieron dos hiperparámetros por índice (72 configuraciones por colección):

Familia	Implementación	Barrido (estructural × de consulta)
LSH	`faiss.IndexLSH` + reordenamiento exacto (`IndexRefineFlat`)	`nbits` $\in \{64 \dots 512\}$ × `k_factor` $\in \{1 \dots 100\}$
IVF-PQ	`faiss.IndexIVFPQ`, `nlist` $= 2\sqrt{N}$, 8 bits por subvector	`M` $\in \{10, 20, 50\}$ B/vector × `nprobe` $\in \{1 \dots 128\}$
HNSW	`hnswlib`, `ef_construction` $= 200$, `space='ip'`	`M` $\in \{8 \dots 48\}$ × `ef_search` $\in \{10 \dots 640\}$

Las 1 000 consultas se ejecutan una a una en **un solo hilo** (Apple M5). Se reportan la latencia media y el p95, y la memoria como el tamaño del índice serializado, que incluye los vectores originales cuando el índice los guarda.

2. Compromiso exactitud–tiempo–memoria

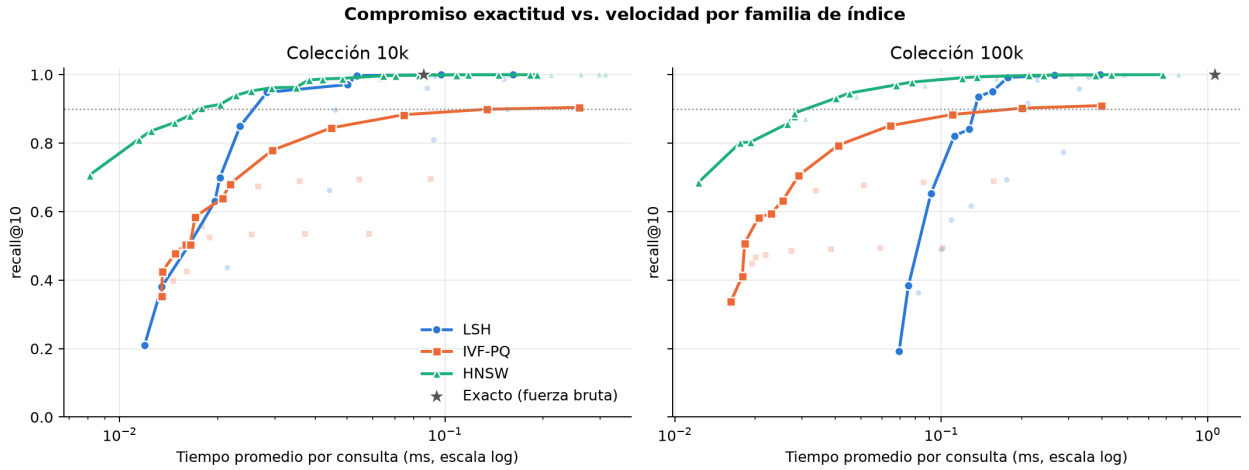


Figura 1: Frontera de Pareto recall@10 vs. latencia por familia. Los puntos tenues son configuraciones dominadas, la estrella es la búsqueda exacta y la línea punteada el requisito 0.90.

HNSW domina la frontera en ambas colecciones. En 100k alcanza recall 0.970 en 0.068 ms y 0.999 en 0.38 ms, frente a 1.06 ms de la fuerza bruta. A cambio, es el que más memoria usa: con $M = 32$ el grafo añade ≈276 B por vector (64.5 MB contra 38.1 MB de los datos crudos) y el que más tarda en construirse (4.3 s). **IVF-PQ es el más compacto.** Con $M = 50$ ocupa 5.9 MB, 6.5× menos que los datos y 11× menos que HNSW. Pero la cuantización impone un **techo de recall que no depende de** `nprobe`: ≈0.49 con $M = 10$, ≈0.69 con $M = 20$ y ≈0.91 con $M = 50$. Las distancias ADC no alcanzan a separar vecinos tan cercanos como los de GloVe. **LSH necesita los vectores originales.** Con solo el código binario (`k_factor` $= 1$) llega a 0.62 con 512 bits. Para superar 0.90 debe reordenar candidatos con los vectores `float32`, y entonces su memoria (44.4 MB) supera a la de los datos. Además, `IndexLSH` recorre todos los códigos, así que su costo sigue siendo $O(N)$.

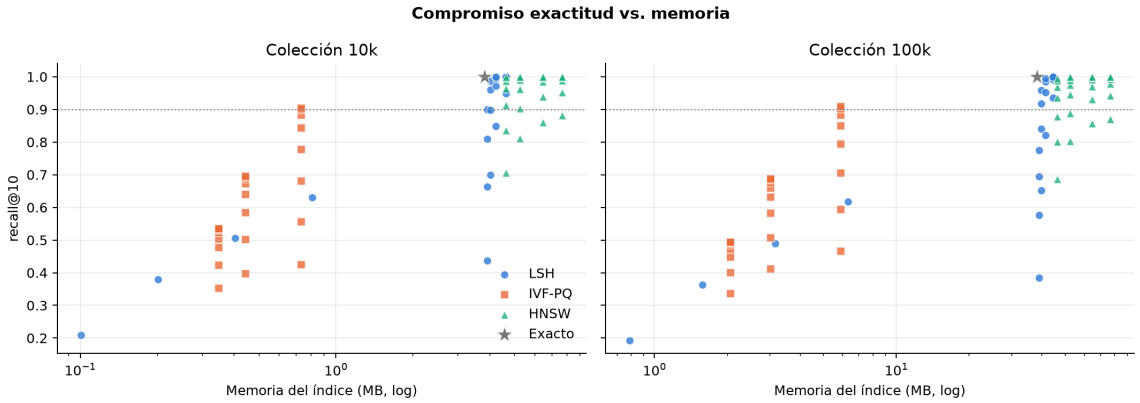


Figura 2: Recall@10 vs. memoria del índice (escala log). IVF-PQ es la única familia que cumple 0.90 por debajo del tamaño de los datos crudos (estrella).

3. Punto de operación: recall@10 $\geq$ 0.90 al menor costo

Escenario: buscador semántico en línea (sugerencia y expansión de términos) con una consulta por petición. **Requisito:** recall@10 $\geq$ 0.90. **Criterio:** entre las configuraciones que cumplen, la de menor latencia media; si hay empate de ± 5 %, la de menor memoria.

Col.	Índice	Configuración	recall	ms (p95)	MB	Constr. (s)	Acel.
10k	Exacto	fuerza bruta	1.000	0.086 (0.114)	3.81	—	1.0×
10k	HNSW	`M=16, ef=20`	0.904	0.018 (0.023)	5.24	0.21	**4.8**×
10k	LSH	`nbits=512, kf=5`	0.948	0.028 (0.030)	4.62	0.01	3.0×
10k	IVF-PQ	`M=50, nprobe=128`	0.904	0.258 (0.298)	**0.73**	0.81	0.3×
100k	Exacto	fuerza bruta	1.000	1.063 (1.325)	38.15	—	1.0×
100k	HNSW	`M=32, ef=20`	0.931	0.040 (0.055)	64.50	4.29	**26.5**×
100k	LSH	`nbits=512, kf=5`	0.936	0.138 (0.145)	44.45	0.06	7.7×
100k	IVF-PQ	`M=50, nprobe=64`	0.902	0.200 (0.211)	**5.88**	1.82	5.3×

Cuadro 1: Puntos de operación por menor latencia (`results/operating_points.csv`).

Justificación con los datos.

- **HNSW (100k).** `M=32, ef=20` es la configuración más rápida que cumple (0.931 en 0.040 ms). Con menos aristas, `M=16, ef=20` se queda en 0.888. La opción que cumple con menos memoria (`M=8, ef=80`, 46.3 MB) es un 20 % más lenta.
- **LSH.** Con 512 bits basta reordenar 5×10 candidatos (0.936). Con 256 o 128 bits hacen falta `k_factor` de 20 o 50 y la latencia sube a 0.156 y 0.211 ms.
- **IVF-PQ.** Solo $M = 50$ cumple, y por poco: con `nprobe=32` se queda en 0.884. En 10k necesita `nprobe=128` de 200 celdas, es decir, recorrer el 64 % de la base; por eso resulta 3× *más lento* que la fuerza bruta. Con pocos datos, el costo fijo de las tablas ADC y del cuantizador grueso no se amortiza.

4. Escalabilidad: de 10k a 100k vectores (datos $\times 10$)

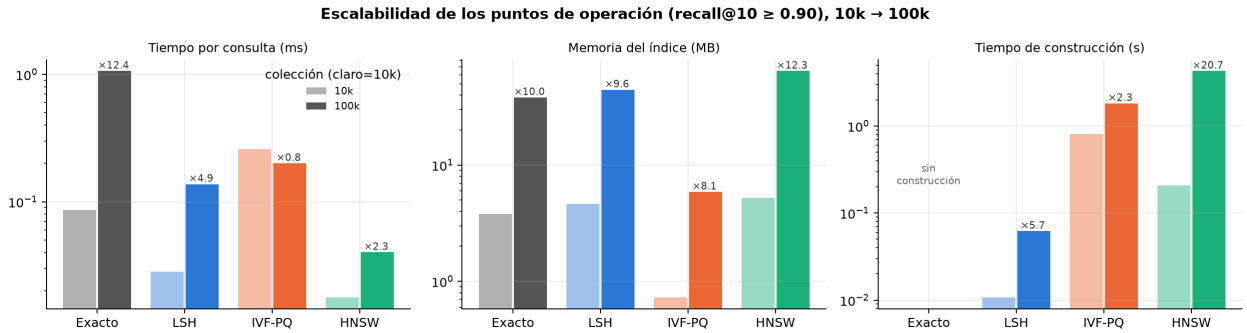


Figura 3: Latencia, memoria y construcción de cada punto de operación en 10k (tono claro) y 100k.

Tiempo. La fuerza bruta crece ×12.4, en línea con su costo $O(N \cdot d)$. HNSW crece solo ×2.3, un comportamiento cercano al logarítmico, y su ventaja sobre el exacto sube de 4.8× a 26.5×. LSH crece ×4.9, porque el escaneo de códigos es lineal. IVF-PQ incluso *baja* (×0.78): `nlist` crece con $\sqrt{N}$ y basta explorar una fracción menor de la base (`nprobe`/`nlist` pasa del 64 % al 10 %). **Hiperparámetros.** Para sostener el requisito, HNSW tuvo que pasar de $M = 16$ a $M = 32$: con la configuración de 10k, en 100k cae de 0.904 a 0.888. LSH conservó la suya (0.948 $\rightarrow$ 0.936).

Memoria. Todo crece casi linealmente: el exacto ×10, LSH ×9.6, IVF-PQ ×8.1 (los centroides son un costo fijo) y HNSW ×12.3, en parte por el cambio a $M = 32$. Al escalar, **el tiempo favorece a HNSW y la memoria a IVF-PQ**.

5. Recomendación para producción

Escenario	**Índice recomendado**	**Por qué (datos medidos en 100k)**
Baja latencia, RAM suficiente (escenario principal)	■ HNSW, `M=32`, `ef_search=40`	`ef=20` ya cumple (0.931), pero conviene margen frente a la variación entre consultas: `ef=40` da 0.970 en 0.068 ms, 15.7× más rápido que el exacto. `ef` se ajusta en caliente sin reconstruir el índice. Costo: 64.5 MB (1.7× los datos).
Memoria limitada (*edge*, millones de vectores)	■ IVF-PQ, `M=50`, `nprobe=64`	5.9 MB (6.5× menos que los datos) con recall 0.902 en 0.20 ms. Su techo de ≈0.91 puede subir con OPQ o reordenando los candidatos con vectores en disco.
No recomendado como índice principal	■ LSH (`IndexLSH` + *refine*)	Es el más barato de construir (0.06 s) y sirve para deduplicación o *streaming*, pero ocupa más memoria que los datos y su latencia crece linealmente con N.

Conclusión. Ninguna familia gana en los tres ejes: HNSW compra velocidad con memoria, IVF-PQ compra memoria con exactitud y LSH solo alcanza el requisito si guarda los vectores originales. Para el escenario definido (recall@10 ≥ 0.90 y baja latencia) recomiendo **HNSW**, cuya ventaja crece con el tamaño de la colección: pasa de 4.8× a 26.5× al multiplicar los datos por 10. Un punto de operación válido en 10k no se hereda sin más, así que los hiperparámetros deben recalibrarse con el tamaño real de producción. **Limitaciones:** una sola máquina y un solo hilo; la construcción multihilo de HNSW no es determinista (el recall varía $\approx \pm 0.005$ entre ejecuciones); la memoria se midió como tamaño serializado y no como RSS del proceso.